

# NumPy & ndarray Notes

*Core Concepts: Array Objects, Homogeneity vs. Heterogeneity, Vectorized Arithmetic, and Multi-Dimensional Matrices*

## 1. Introduction: The NumPy Array as a Specific Object

Recall from our previous notes on Classes and Objects that in Python, 'everything is an object'. Every single entity has an underlying class blueprint that defines its internal properties (data members) and functionalities (methods). When we work with scientific and mathematical computing, standard Python lists become cumbersome and inefficient. This is where NumPy comes in.

When you create an array in NumPy using `np.array(...)`, what you get back is a specific kind of object called an `ndarray` (an instance of the `numpy.ndarray` class). Just like our earlier `Student` or `buggati_chiron` objects, this `ndarray` object encapsulates specific data members (like shape, dimension, and data type) and provides hundreds of specialized methods to manipulate numerical data at blazing speeds.

## 2. Python Lists vs. NumPy Arrays: Heterogeneous vs. Homogeneous

The fundamental structural difference between a standard Python list and a NumPy array is how they store data in computer memory:

- **Python Lists are Heterogeneous:** A regular Python list is actually a dynamic array of pointers (memory references). Because it stores pointers rather than the raw data itself, a single Python list can hold mixed data types simultaneously—such as `[10, 'mike', 3.14, True, [1, 2]]`. However, storing separate object headers, reference counts, and scattered memory addresses for every single element introduces huge memory overhead and slow iteration.
- **NumPy Arrays are Homogeneous:** A NumPy array strictly stores elements of the exact same data type (e.g., all 64-bit integers, or all 64-bit floats). Under the hood, the raw bytes are packed directly next to each other in a single contiguous block of C-style memory. There are no individual Python object wrappers around each number, which makes NumPy arrays extremely memory-efficient and blazingly fast to compute on.

**Why Homogeneity Makes NumPy Efficient:** Because every element has the exact same byte size and data type in memory, the CPU can jump directly to any element in  $O(1)$  time and perform bulk vector computations (SIMD) without needing Python type-checking at every single step.

### Automatic Type Upcasting (Type Coercion)

If you pass a Python list containing mixed data types into `np.array()`, NumPy will not throw an error. Instead, it will automatically 'upcast' all elements to the lowest common data type so that homogeneity is strictly preserved across the entire array.

```
import numpy as np

# 1. Standard Python list with heterogeneous types:
py_list = [10, 3.14, "hello"]
print("Python list:", py_list) # Output: [10, 3.14, 'hello'] (mixed types preserved)

# 2. NumPy array with mixed integers and floats -> Upcasts all to float:
arr_float = np.array([1, 2.5, 3])
print("NumPy float array:", arr_float) # Output: [1.  2.5  3. ]
print("Array data type:", arr_float.dtype) # Output: float64
```

```
# 3. NumPy array with integers, floats, and strings -> Upcasts everything to string:
arr_str = np.array([1, 2.5, "three"])
print("NumPy string array:", arr_str)    # Output: ['1' '2.5' 'three']
print("Array data type:", arr_str.dtype) # Output: <U32 (Unicode string)
```

### 3. Arithmetic Operations: Vectorization vs. List Manipulation

In standard Python, the basic arithmetic operators applied to lists do not execute mathematical calculations on the numbers. Instead, they perform sequence operations on the list container itself:

- **List Addition (`+`)**: Concatenates / glues the two lists together into one longer list (`[1, 2] + [3, 4] -> [1, 2, 3, 4]`).
- **List Multiplication (`\*`)**: Repeats the entire list multiple times (`[1, 2] \* 3 -> [1, 2, 1, 2, 1, 2]`).
- **List Subtraction / Division**: Throws a `TypeError` (Python does not support subtraction or division on lists).

In contrast, NumPy arrays natively support element-wise mathematical operations directly without having to write explicit `for` loops. This core feature is called vectorization.

```
import numpy as np

# --- Standard Python Lists ---
l1 = [1, 2, 3]
l2 = [4, 5, 6]
print("List + List (concatenation):", l1 + l2)    # [1, 2, 3, 4, 5, 6]
print("List * 3 (repetition):", l1 * 3)          # [1, 2, 3, 1, 2, 3, 1, 2, 3]
# print(l1 * l2) <-- Raises TypeError! Cannot multiply sequence by non-int

# --- NumPy Arrays (Element-wise Vectorized Math) ---
a1 = np.array([1, 2, 3])
a2 = np.array([4, 5, 6])

print("Array Addition (a1 + a2):", a1 + a2)      # [5, 7, 9]    (1+4, 2+5, 3+6)
print("Array Subtraction (a2 - a1):", a2 - a1)   # [3, 3, 3]    (4-1, 5-2, 6-3)
print("Array Multiplication (a1 * a2):", a1 * a2) # [4, 10, 18]   (1*4, 2*5, 3*6)
print("Array Division (a2 / a1):", a2 / a1)      # [4.  2.5  2. ]   (4/1, 5/2, 6/3)
print("Scalar Multiplication (a1 * 10):", a1 * 10) # [10, 20, 30]
print("Exponentiation (a1 ** 2):", a1 ** 2)      # [1, 4, 9]
```

**The Magic of Vectorization:** Vectorized operations run directly in pre-compiled C routines under the hood. This completely avoids Python loop overhead and achieves performance improvements of 50x to 100x on large datasets.

### 4. Multi-Dimensional Arrays (ndarray) & Strict Matrix Integrity

In standard Python, a 2D matrix is typically represented as a nested list (a list of lists), such as `[[1, 2, 3], [4, 5, 6]]`. NumPy formalizes multi-dimensional data into the `ndarray` object with a very important mathematical rule:

- **Strict Rectangular / Grid Structure:** In a multi-dimensional NumPy array, every row must have the exact same number of columns, and every column must have the exact same number of rows. All elements in the matrix must exist! Unlike

standard Python (which allows 'ragged' or 'jagged' lists where row 0 has 3 items and row 1 has only 1 item), an ndarray represents a fully uniform, rectangular N-dimensional grid.

## Key Data Members (Attributes) of an ndarray Object

Connecting back to our OOPS concepts, an `ndarray` object has several key data members (attributes) that tell you everything about its internal structure:

- **.shape:** A tuple representing the size along each dimension (e.g., `(2, 3)` indicates 2 rows and 3 columns).
- **.ndim:** The total number of dimensions/axes (`1` for a 1D vector, `2` for a 2D matrix, `3` for a 3D volume/tensor).
- **.dtype:** The precise data type of the homogeneous elements stored inside (e.g., `int64`, `float32`).
- **.size:** The total count of all elements across all dimensions (e.g., for shape `(2, 3)`, size is `6`).

```
import numpy as np

# 1-Dimensional Array (Vector)
vec = np.array([10, 20, 30, 40])
print("1D Array:", vec)
print("Shape:", vec.shape)    # Output: (4,)
print("Dimensions:", vec.ndim) # Output: 1

# 2-Dimensional Array (Matrix) - Uniform rectangular grid
mat = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print("\n2D Matrix:\n", mat)
print("Shape (rows, cols):", mat.shape) # Output: (2, 3) -> 2 rows, 3 columns
print("Dimensions:", mat.ndim)         # Output: 2
print("Total element count:", mat.size) # Output: 6
print("Element data type:", mat.dtype)  # Output: int64

# Strict Matrix Rule:
# If you try to create a jagged nested list like [[1, 2, 3], [4, 5]],
# NumPy cannot treat it as a standard 2D numerical matrix because all
# elements in the matrix grid must exist uniformly.
```

## 5. Summary Comparison: Python List vs. NumPy ndarray

| Feature                  | Python Standard List                         | NumPy ndarray                                    |
|--------------------------|----------------------------------------------|--------------------------------------------------|
| <b>Data Types</b>        | Heterogeneous (mix int, float, str, objects) | Homogeneous (all elements strictly same type)    |
| <b>Memory Layout</b>     | Array of pointers to scattered objects       | Single contiguous memory block in C              |
| <b>Arithmetic (+, *)</b> | + concatenates lists, * repeats list         | + and * execute element-wise math (Vectorized)   |
| <b>Matrix Structure</b>  | Can be jagged / ragged (uneven inner lists)  | Strict N-D grid (all matrix elements must exist) |